The Critical-Projects List Contains Kubernetes and Misses zlib

Dependency concentration as a criticality signal

Author. Chris Brink (independent) **Version.** Draft v0.17, 2026-10-19 (v0.2: retitled; artifacts section. v0.3: package-versus-repository distinction made explicit; recommendation section ends on the artifact; CLI gained direct Cargo.lock support. v0.4: tool and rankings split to their own repo, github.com/thefalsework/conemass, with commit history intact; the tool is named conemass — the metric keeps the name ORACLE in this text. v0.5: registered RustSec pooled-retrodiction result added to limitations — ORACLE lost the pooled cell to dependent count, reported in full. v0.6: this copy ships with the tool at github.com/thefalsework/conemass, which is now the paper's canonical home; the research repository retains the study scripts and the archival record. v0.7, 2026-09-05: the metric is named conemass throughout this text and in the tool's CSV columns; ORACLE remains its working name in the frozen registered studies and earlier archived versions. Pure rename — no computation, ranking, or data row changed, as the git history of the published CSVs shows. v0.8, 2026-09-22: closest-prior-work note added, positioning conemass against Pfeiffer's PageRank-plus-truck-factor approach (MSR 2021); shell-dependent gameability added to limitations. v0.9, 2026-09-23: evidentiary status of the title's two halves made explicit — the zlib half is the measured claim, the Kubernetes half a category observation, since Kubernetes has no node in the corpora studied and the symmetric test remains not run; also, the unicode-ident dependent-count tie convention is now stated in the text (average-rank 3,582 in the study scripts vs min-rank 3,304 in the published CSVs, previously reconciled only in rankings/README.md). No computation or ranking changed. v0.10, 2026-09-23: head-overlap section added with measured top-40/top-100 overlap against dependent count and PageRank on both corpora (`06-head-overlap.mjs`), including the facts that cut against us: PageRank places unicode-ident #1 on crates and liblzma5 at 36 on Debian — the differentiation against PageRank is narrower than against dependent count, and is now stated with numbers rather than left for a reader to discover. v0.11, 2026-09-23: Result 4 added — registered vintage-trajectory study (`07-drift.mjs`) across the ten dated Debian snapshots: liblzma entered the archive at #8 and never climbed, so the monitorable signal is arrival-into-the-head, not drift; an earlier drift-as- climbing framing used in outreach is corrected here with the measurement that killed it. v0.12, 2026-09-23: the registered succession-filter follow-up was run (`08-arrival-filter.mjs`) and Result 4's churn paragraph now carries the measured verdict — median 13 genuine arrivals per release, monitorable under the registered threshold. v0.13, 2026-09-23: the arrival pattern tested on the second ecosystem (`09-crates-arrival.mjs`, four dated crates snapshots): replicates — unicode-ident absent until it did not exist, then #2 on arrival, inheriting the literal seat of unicode-xid — while top-100 monitorability does NOT transfer to a hypergrowth registry (median 35 genuine arrivals per step, over the registered dead threshold; reported, not patched). v0.14, 2026-09-23: graph-anatomy study (`10-graph-anatomy.mjs`, fifteen graphs, four registered shape predictors): three predictors failed including the registered primary; mean truncated-cone depth survived (rho -0.73, correct family ordering) and is stated as a candidate with a disclosed unresolved band, not a law. v0.15, 2026-09-23: the Mathlib declaration-graph study — registration, verification gate, null analysis, rankings, and the depth measurement this paper's anatomy section cites — is published at `examples/mathlib/` in the conemass repository; it had been complete but local, which contradicted the artifact promise below. No number changed. v0.16, 2026-09-23: corrections from an external review that checked this paper's claims against the published artifacts. (a) The anatomy section's Mathlib verdict mixed measures — "coincides" was a Spearman fact while the study's registered outcome is top-40 overlap, on which Mathlib (21/40) separates like Debian (22/40); the out-of-family point is consistent with the depth candidate, not against it, and the section now says so with the original wording quoted. (b) Cycle handling disclosed in the artifacts section: the tool condenses cycles, so a cycle donates one unit total, a departure from the printed formula. (c) "Two ecosystems, same law" reworded to match the paper's own candidate-not-law discipline. (d) The version header had been left at v0.9 while the changelog grew; the stale study count in the artifacts section (six, actually eleven) corrected; the archival DOI's "this repository" label clarified. The same review found two false claims in the Prove2Me example's report and scheduler spec (bracket direction and score exactness), corrected there with a dated section

and a corrected open-only unlock ranking (`examples/prove2me/unlock-open.mjs`). (e) Same day, second pass: this paper itself repeated the exactness claim twice — "exactness theorems on AND-semantics graphs" in Result 3 and "AND-graph exactness" in the limitations — although the repo contains no such theorem. Both now state what is true: on AND-semantics graphs the released set is enumerable and checkable; the mass is a fractional share over it, never exact. v0.17, 2026-10-19: two external ground-truth tests added after the studies above, one against the paper. (a) A citation graph (`examples/citations/`, 42,796 papers through 2012 scored against 2013–2020 citations) falsifies the anatomy section's depth candidate at an out-of-family point — depth 3.44 predicted coincidence with PageRank, measured overlap 7/40 — and shows why: 79% of cones are truncated at the cap, against 6.5% on Debian and 1.0% on crates, so the ranking is the cap over age. The precondition the paper never stated (bounded closures; the truncated-cone fraction) is now stated in the anatomy section and the limitations, the cap-robustness claim in Result 1 is qualified to package graphs, and the CLI reports the fraction on every run. (b) The repository's Alpha-Omega report had claimed flat SAST yield in rank; at population scale yield falls toward the top of the ranking, and the claim was withdrawn there on 2026-10-18. This paper made no yield claim; the limitations now carry the correction and the result that survived (severity unrelated to reach). No number in Results 1–4 changed. All computations cited here are committed with their code and raw output in `oracle-scanner/` at github.com/thefalsework/papers — except the Mathlib declaration-graph study, which lives at `examples/mathlib/` in this repository alongside the other per-corpus applications; each script states its expectations in a header written before the run and its results in a dated postscript. Code is Apache-2.0; text is CC-BY-4.0. Archived: the research-record repository (thefalsework/papers, study scripts and preprint source) at DOI 10.5281/zenodo.22261990; the conemass tool and dated rankings — this repository — at DOI 10.5281/zenodo.22261985; both also at Software Heritage.

Summary

The OpenSSF criticality-score top-1000 — the list consumed by the Securing Critical Projects working group — contains Kubernetes and misses zlib. The two halves of that sentence carry different kinds of support, and the difference should be on the table from the start. The zlib half is the measured claim this piece documents. The Kubernetes half is a category observation, not a conemass comparison: Kubernetes is an intensively monitored application that no package depends on — it has no node in the dependency graphs studied here — sitting at the head of a list whose purpose is to surface unwatched load-bearing dependencies. We have not shown Kubernetes scores low on conemass (see the symmetric- test limitation below); we observe that it is the wrong *kind* of thing to top such a list. The incumbent also misses serde, syn, proc-macro2, libexpat, and libxml2, and its collection pipeline, which enumerates GitHub-hosted source repositories, could not have ranked the xz project at any position in 2022, because xz was hosted elsewhere. One distinction runs through this piece and is best fixed now: the metric we describe ranks *shipped packages*, the objects dependency graphs contain — the Debian binary package liblzma5, the crate unicode-ident — while the incumbent ranks *upstream source repositories*. Our comparison maps packages to their upstream repositories by hand, and the two xz facts below are facts about two different objects: the package shipping the backdoored code ranked eighth in its archive by concentration, and the project's repository sat entirely outside the incumbent pipeline's coverage. We describe a complementary metric, computable from a dependency graph alone, whose top ranks are precisely the packages the incumbent misses: on the last Debian release before the xz backdoor, it ranks liblzma5 eighth in the archive, against #173 by dependent count; on crates.io it ranks unicode-ident — six direct dependents, one maintainer, present in nearly every Rust build — second in the registry, against #3,582 by dependent count. (Tie convention: thousands of crates share that six-dependent count; the study scripts rank ties by average position, giving 3,582, while the published CSVs in the repository use minimum position, giving 3,304. Same data, same six dependents; conemass rank is #2 under both.) The metric is not a replacement for criticality scoring. It measures a different quantity — load rather than fame — and the two disagree exactly where prioritization mistakes are most expensive.

The metric

For a node x in a dependency graph, define

$$\text{conemass}(x) = \Sigma\ 1/|\text{cone}(u)|,$$

summed over all packages u whose truncated transitive-dependency set ("cone," breadth-first, capped at 200 nodes) contains x. In words: count every toolchain that x is part of, weighting each by the reciprocal of its size. A package that constitutes half of ten small toolchains scores higher than a package that is a thousandth of ten thousand large ones, at identical dependent counts. The quantity measured is *concentration of reach*.

Two properties matter for this audience. First, the metric requires only the dependency graph — no stars, contributors, commit activity, funding data, or hosting-platform API access — so a registry operator can compute it for an entire ecosystem in minutes (our runs: 63,436 Debian packages or 84,439 crates, single-threaded, under fifteen seconds each). Second, it is structurally different from every input the incumbent uses: dependent counts, PageRank-style measures, and the criticality score's signals all reward volume or visibility, and rank-correlate accordingly.

The closest prior work is Pfeiffer (MSR 2021),[2] which makes the same diagnosis of the incumbent — popularity over criticality — and identifies quiet-critical packages (six, idna) via PageRank on the reversed dependency graph combined with low truck factor. conemass shares the diagnosis and the goal but is a different functional: harmonic cone-membership mass rather than random-walk centrality. How far the two separate in practice is measured, not asserted — per-corpus head-overlap numbers, including the cases that favor PageRank, are in the head-versus-bulk section below — and Pfeiffer's maintainer-surface pairing composes with conemass exactly as it does with PageRank, for anyone who wants a risk-to-capacity ratio.

The functional was originally derived as the closed-form expected-gain law of a synthetic graph-growth model, where it provably and completely accounts for growth differences that degree, age, PageRank, k-core, and exact transitive-dependent counts cannot express. That derivation is documented elsewhere in the repository; nothing in this piece depends on it. Here the metric is offered as a ranking, and a ranking is judged by what it flags.

Data and vintage

Three datasets, and the dating is the methodological core of the piece:

- **Debian**, main/binary-amd64 dependency graphs for the ten stable releases 2007–2025, extracted from archived Packages files with parsing choices fixed in the extraction script. The retrodiction below uses **bookworm (2023) — the last stable release before CVE-2024-3094 was disclosed in March 2024.**
- **crates.io**, the 2022 dependency snapshot from the same repository's earlier growth studies.
- **The incumbent list**: the "1000 critical projects" CSV produced by the OpenSSF criticality-score pipeline (Pike scoring), **June 2022 vintage**, retrieved from a Scorecard maintainer's archival repository.[1]

The June 2022 list and the 2022 crates snapshot are contemporaneous, and both predate the public disclosure of the xz backdoor by roughly 21 months. No ranking reported here can be contaminated by post-incident attention: the stars, mentions, and activity that xz, liblzma, and unicode-ident accumulated after March 2024 do not exist in any of this data. We did not select an old list to disadvantage the incumbent; we selected the list that removes hindsight from both sides of the comparison.

Result 1: the xz retrodiction

On Debian bookworm, one release before disclosure:

package	conemass rank	dependent-count rank	PageRank rank
liblzma5	**8** / 63,436	173	36
libgcrypt20	49	150	151
libexpat1	38	102	56

package	conemass rank	dependent-count rank	PageRank rank
zlib1g	4	7	8
libssl3	18	19	35

The pattern, not the single number, is the result. Packages that are famous as well as load-bearing (zlib, OpenSSL) rank high on every metric; the metrics agree where fame is deserved. They diverge on the quiet rows: liblzma, libgcrypt, and libexpat — each a small, low-visibility library with a history of under-resourcing — move up by one to two orders of magnitude under concentration weighting. The liblzma ranking is robust to the one free parameter: at cone caps of 50, 100, 200, 400, and 800 its rank is #8 in every case, and the archive's conemass top-100 overlaps 92–99% between adjacent caps. (Added 2026-10-19: that robustness is a property of package graphs, where few cones reach the cap. On a graph whose closures exceed it almost everywhere the cap is the ranking; measured on a citation graph in the limitations below, where top-40 overlap with citation count fell from 13 to 3 as the cap rose from 50 to 3000.)

Transitive-dependent *counts*, for comparison, are unusable at the head of the distribution: hundreds of major libraries saturate the cap and tie. Concentration weighting is what separates them.

Result 2: crates.io, and a specific threat class

The same computation on the 2022 crates.io graph:

crate	conemass rank	dependent-count rank	direct dependents
libc	1 / 84,439	14	4,330
unicode-ident	**2**	3,582	6
proc-macro2	3	17	3,392
quote	4	16	4,135
syn	5	15	4,202
cfg-if	6	52	835
serde	7	1	16,012

unicode-ident is the illustrative row: a single-maintainer crate with six direct dependents that reaches nearly every Rust build through the proc-macro2/syn chain. Dependent-count scoring places it below three and a half thousand other crates. Concentration places it second in the registry. This is the same profile as liblzma — minimal direct visibility, near-total indirect presence — identified by the same computation in an unrelated ecosystem.

The extreme divergers form a coherent class rather than noise. Sorting the conemass top-1000 by how much worse their dependent-count rank is yields, almost without exception, degree-one procedural-macro companion crates: openssl-macros, wasm-bindgen-macro, pin-project-internal, darling_macro, and so on. Each has exactly one direct dependent (its parent crate) and is present in every toolchain its parent reaches. Procedural macros execute at build time on developer and CI machines. A package with code execution by design, one direct dependent, near-zero independent scrutiny, and presence in a large fraction of all builds is a supply-chain target profile, and dependent-count scoring cannot surface it even in principle: the count is one.

Head versus bulk: what the ranking actually buys

Global rank correlation between conemass and volume metrics is high — Spearman 0.95–0.99 across full registries — and that number, read alone, invites the wrong conclusion. Nobody consumes rank 40,000 of a criticality list. The artifact a registry operator or working group consumes is the head — a top-40 watchlist, a top-100 review queue — and the head is where the orderings come apart. Measured directly (top-k set overlap, `06-head-overlap.mjs`, run 2026-09-23 on the same corpora as Results 1 and 2):

comparison	Debian 2023	crates.io 2022
conemass vs dependent count, top-40	12/40	19/40
conemass vs dependent count, top-100	35/100	47/100
conemass vs PageRank, top-40	22/40	35/40
conemass vs PageRank, top-100	63/100	75/100

Against dependent count — the signal the incumbent's ecosystem actually consumes, via deps.dev — the head disagreement is the product: two-thirds of the Debian top-40 and half the crates top-40 are rows dependent-count ranking does not surface, and Results 1 and 2 document that those rows are the quiet load-bearing class, not noise.

Against PageRank the honest picture is narrower, and it splits by corpus. On Debian the two genuinely diverge at the head (22/40), and the rows conemass surfaces that PageRank buries are a coherent class: the Kerberos gateway chain (libkrb5support0 at PageRank 263, libk5crypto3 at 345, libkeyutils1 at **981** — all conemass top-30), libcom-err2 (251), libgdbm-compat4 (143, dependent-count rank 3,836). These are deep shared-runtime chains sitting under every cone their gateway reaches — the same topology as the xz attack path. On crates.io, by contrast, PageRank's head nearly coincides with ours (35/40), and the two facts that cut against us most directly are these: PageRank ranks unicode-ident **#1** on crates — more prominent than conemass's #2 — and places liblzma5 at 36 on pre-disclosure Debian, inside a top-40 watchlist (conemass: 8; dependent count: 173). A team already running PageRank on its dependency graph would have had both headline packages on a 40-row watchlist. We are aware of no evidence that anyone was; the incumbent does not use PageRank, and Pfeiffer's proposal (the one published exception) was not adopted.

What conemass offers over PageRank is therefore not, on current evidence, the discovery of rows PageRank cannot see — Debian's gateway-chain class excepted. It is: zero parameters against a tuned damping factor; a deterministic, enumerable score (a package's mass is a finite sum you can list — these cones, this much credit each) where a PageRank value is a fixed point with no operational reading, which matters when a triage decision has to be defended; and, on graphs with AND-semantics (the proof-graph application in this repository), an enumerable dispatch claim — closing this node is required by exactly these listed parents — that a diffusion metric cannot state. (An earlier version of this sentence said "exactness theorems"; there is no such theorem, and the mass itself is not exact — the exactness belongs to the enumerated list, as the dated correction in the Prove2Me report spells out.) Where the two coincide, conemass is the cheaper and more auditable of the pair; where they diverge, the divergence has so far favored the quiet class the paper is about. Both facts are now measured, and the overlap table above is the one a skeptical reader should check first.

Can the divergence be predicted from the graph alone, before running either metric? We tested four registered shape statistics across fifteen graphs (ten Debian snapshots 2007–2025, four crates snapshots 2016–2022, one theorem-dependency graph; `10-graph-anatomy.mjs`, predictors and directions fixed before twelve of the fifteen outcomes were seen). Three failed, including the registered primary — chain fraction, the statistic our own gateway-chain reading suggested, came back near zero (rho −0.14), and two others had the wrong sign. One survived: mean truncated-cone depth (rho −0.73, with the family ordering right — Debian, the separator, is the deep family at ~5; crates and the proof graph sit at 2.5–4 and coincide). The mechanism reading is direct: conemass credit traverses a dependency chain undamped, PageRank decays per hop and splits across links, so the two diverge where cones are deep and agree where the mass sits a hop or two away. An out-of-family check on the Mathlib declaration graph (prediction stated before measuring) was first reported here as a miss: "truncated depth 4.45, inside Debian's range, while Mathlib coincides with PageRank — 4–5 is the disclosed unresolved band." A same-day external review caught that this verdict mixed two measures. The study's registered outcome is top-40 overlap, and on that measure Mathlib is **21/40** — numerically the same head separation as Debian's 22/40 — while "coincides" was a verdict about Spearman over all 308K declarations (0.96). Measured consistently, the Mathlib point (depth 4.45, overlap 21/40) *fits* the depth trend rather than breaking it. What actually distinguishes Mathlib from Debian is the content of the divergence, not its size: Mathlib's conemass-only rows are foundational plumbing rather than quiet risk (its registered finding criterion failed on row quality, with overlap one row over the kill line). The falsifiable statement for the next new graph,

corrected: truncated-cone depth above ~4 predicts head divergence from PageRank, below ~4 predicts coincidence — and whether the divergence is *interesting* is a property of the corpus, not of the shape statistic. One surviving predictor out of four registered, on three graph families plus one consistent out-of-family point, is a candidate, not a law.

A second out-of-family point, added 2026-10-19, goes against the candidate. A citation graph (OpenAlex topic "Neural Networks and Applications", 42,796 papers through 2012, 227,985 citation edges, `examples/citations/` in the conemass repository) has truncated-cone depth **3.44**, inside the crates band (2.3–3.95) where the statement predicts coincidence with PageRank. Measured top-40 overlap is **7/40**, the strongest divergence of any graph in the set. The prediction fails. The reason is visible in a statistic the anatomy study recorded only as an unregistered observation: mean truncated cone size. On this graph it is 179.7 of a cap of 200, and 79% of all cones are truncated; the reference closure of nearly every 2012 paper reaches the 1943 founding paper, so the cap, not the structure, decides which 200 ancestors each paper credits, and citation cycles produce strongly connected components (the largest has 3,403 papers) whose members share one score — one such group ranks a paper with 1,777 citations level with one that has 1. The divergence from PageRank is real and it is noise: with citation count and publication period held fixed, conemass's rank correlation with citations received 2013–2020 is −0.03 to +0.07, the same as PageRank's and reach's. The same saturation statistic sat at 150.4/200 on Mathlib, which coincided, so saturation does not predict the overlap's direction either; what it predicts is that the overlap is not meaningful. The corrected statement, which is a scope restriction made after seeing the data and carries that status: the depth predictor was validated on graphs where the cap rarely binds and says nothing where it binds for most cones. Measured the same way (cap 200, SCC condensation, same traversal; the depths reproduce the anatomy study's to two decimals), the fraction of cones truncated at the cap is **6.5%** on Debian 2023, **1.0%** on crates 2022, and **79.1%** on the citation graph, with largest strongly connected components of 7, 2, and 3,403 nodes. That fraction is the precondition the paper did not state: where it is a few percent the ranking is conemass; where it is most of the graph the ranking is the cap over age, and no shape statistic rescues it. The CLI reports the fraction on every run as of 2026-10-19 and flags it above one half.

Result 3: the incumbent comparison

We mapped the conemass top-10 of crates.io, the watchlists above, and the threat-class list to their 2022 GitHub repositories (hand-curated mappings, published with the code) and checked membership in the incumbent's top-1000.

Of the crates conemass top-10, one appears: libc, at #257. serde — the most depended-upon crate in the registry by raw count — is absent. So are syn, proc-macro2, quote, cfg-if, and unicode-ident. The threat-class list is absent in its entirety, 0 of 7. On the Debian side: OpenSSL is present at #42; zlib, libexpat, and libxml2 are absent.

Two distinct failures produce this, and they should not be conflated.

The scoring function fails on quiet finished infrastructure. The Pike score's inputs are contributor counts, organization counts, commit frequency, release cadence, issue activity, and mention counts. zlib is maintained software in its finished state: one maintainer, low commit frequency, rare releases, few issues. It scores near zero on every input while being, by any reasonable definition, among the most critical code in existence. This failure is intrinsic to fame-and-activity weighting and applies to every package in the tables above.

The collection pipeline fails on anything not hosted on GitHub. xz was hosted at git.tukaani.org in 2022. The pipeline enumerates GitHub repositories, so xz could not have appeared at any rank regardless of the scoring function. This is a coverage limitation, not a scoring limitation; it would have excluded xz even under a perfect score. We note it separately because it was the binding failure for the one package everyone in this field agrees was the catastrophe.

Result 4: ten releases of vintage — load arrives, it does not climb

If concentration is a warning signal, the operational question is *when it appears*. The same computation was run on all ten archived Debian stable releases, 2007–2025, with the expectation registered before the run (`07-drift.mjs`): that liblzma's load predated the takeover and would show no climb during the attacker's window. It holds, more strongly than guessed:

release	2007	2009	2011	2013	2015	2017	2019	2021	2023	2025
liblzma conemass rank	—	—	**8**	8	8	6	8	10	8	19

liblzma entered the archive at **#8 on arrival** — squeeze, 2011, the release where dpkg adopted xz compression — and sat pinned in the top ten for fourteen years. There is no takeover-window movement at all (2019 → 2023: 8 → 10 → 8). The attacker did not add a single edge to this graph; Debian's own adoption decision built the position in 2011, and the attacker selected a package whose blast radius already existed. Two consequences:

The monitorable signal is arrival, not drift. A "watch packages climbing the rankings" alert — a framing we ourselves used in early outreach — is measured here and killed: rank trajectories at the head are nearly flat between releases. What the trajectory data does support is an *arrival* alert: a new package entering the top ranks directly, which is the moment load is created and the moment scrutiny is furthest behind it. That alert would have fired on the entire compression-library class years before any incident: liblzma (2011, on arrival), lz4 (2017, from rank 1,482), zstd (2019, from 1,243) — and on libkeyutils1 (2009, from 989), the Kerberos-chain package that the head-versus-bulk section shows PageRank buries at 981.

The instrument also records the remediation. liblzma5's one large move in eighteen years is the drop from 8 to 19 in trixie (2025) — the ecosystem visibly de-concentrating the xz path after the backdoor. A metric that shows both the exposure forming and the fix landing is a metric an operator can watch.

The pattern replicates on the second ecosystem. The same computation on four dated crates.io snapshots (2016, 2018, 2020, 2022; `09-crates-arrival.mjs`, expectations registered before the run): unicode-ident is absent from every snapshot before it existed, then **#2 of 84,439 on arrival** — one step, on the ecosystem's adoption decision, when proc-macro2 and syn swapped it in during 2022. The seat is literal: unicode-xid, the crate it displaced, ran 158 → 12 → 2 across 2016–2020 and fell to 135 in 2022; the #2 load position changed occupants, not size. Nor is it an isolated case — the entire macro toolchain arrived as a block in 2018 (proc-macro2 absent → 13, syn absent → 15, quote absent → 16), then sat. Two ecosystems, same pattern — load arrives, it does not climb — measured twice, which by this paper's own discipline makes it a replicated observation, not a law. One caveat recorded rather than discovered: a dependency *swap* changes the package name, so the succession filter below does not absorb it — the new occupant fires as a genuine arrival. That is the correct behavior; "a crate that did not exist a year ago now holds the #2 load position previously held by a watched crate" is precisely the alert an operator wants loudest.

Stated against the registered thresholds: raw top-100 churn between adjacent releases has a median of 23 new entrants, which fell in the registered grey zone (≤15 monitorable, ≥30 dead). The entrant lists are dominated by mechanical version successions (`gcc-N-base`, `libicuNN`, `python3.N`, the 2025 t64 ABI renames), so the registered follow-up (`08-arrival-filter.mjs`) applied a name-succession filter — normalize names by stripping digits and ABI suffixes; an entrant whose stem was already in the previous top-100 is a succession, not an arrival — and re-counted. Result: **median 13 genuine arrivals per release** (range 8–19), under the ≤15 threshold; the verdict closes as monitorable. The filter confirms every known case at its date (liblzma 2011, lz4 2017, zstd 2019, keyutils 2009, the Kerberos stack 2011) and correctly absorbs the renames, including `libkrb53` → `libkrb5-3`. For an entire OS distribution, that is roughly seven alert rows per year. The genuine-arrival lists still carry payload-implausible rows (doc and font packages); filtering those is a product decision, not a metric one, and is left visible. On crates.io the monitorability verdict goes the other way and is reported as registered: median 35 genuine arrivals per two-year step (47 → 35 → 25 across the window), over the ≥30 dead threshold — top-100 arrival alerting is too noisy on a registry that doubled in size every two years of the sample. The churn there is real growth, not renames (the succession filter removed zero rows; crates version inside one name), and it declines monotonically as the registry matures. A growth-adjusted or narrower head would need its own registered thresholds and has not been run. So the deployment claim is scoped honestly: arrival-into-the-head is

the signal on both ecosystems; the alert volume is proven manageable on a curated distribution and not yet on a hypergrowth open registry.

Limitations

Stated in full, because the comparison above is one-directional and the piece is descriptive throughout.

- **We did not run the symmetric test.** We checked whether conemass's head appears in the incumbent's list; we did not systematically check whether the incumbent's head (Linux, git, Node, Kubernetes) scores low on conemass. Most of the incumbent's top entries are applications rather than packages and have no node in a package dependency graph, so the symmetric test requires a corpus-mapping exercise we have not done. Until it is done, the correct statement is that conemass's head is invisible to the incumbent — not that the two rankings are anti-correlated. This is also why the title's Kubernetes half is a category observation rather than a measured comparison, as stated in the summary: Kubernetes has no node in either published corpus (neither string matches any row of the Debian or crates top-1000s, and Debian does not package it), so conemass can say nothing about its rank, only that a dependency-graph metric cannot see it at all — which is, in miniature, the difference between the two instruments.
- **One retrodiction is one retrodiction.** liblzma at #8 is a single post-hoc case, chosen because it is the consensus catastrophe. The metric's forward value is untested. The honest deployment model is a standing ranking whose future hits and misses accumulate in public.
- **A registered pooled retrodiction against RustSec advisories went against us, and we report it.** After this piece was drafted we ran the natural stress test: metrics computed on the 2022 crates snapshot, labels = RustSec vulnerability advisories dated strictly after it (159 affected crates in-snapshot), share of labeled crates in each metric's top decile. Every graph metric crushes the random null (55–63% vs 10%), but conemass (0.554) **lost** to dependent count (0.629) and PageRank (0.598) at the registered headline cell. Label bias favors popular crates (advisories are filed where the scrutiny is) and was stated before the run — but the registration committed to reporting the outcome as a loss, so: at *pooled* advisory retrodiction, concentration does not beat volume. This piece's claim is complementary value at the divergent head, which is a different cell and remains untested either way; a reader should still know the pooled test went the other way (`05-rustsec.mjs`, registered, single run).
- **Global rank correlation with volume metrics is high** (Spearman 0.95–0.99 across full registries). The divergence is concentrated at the head of the ranking; the head-versus-bulk section gives the exact per-corpus overlap numbers, and a reader should not picture two unrelated orderings.
- **PageRank catches the headline rows too.** As measured in the head-versus-bulk section: unicode-ident is PageRank #1 on crates and liblzma5 is PageRank 36 on pre-disclosure Debian. The claim this paper can support against PageRank is the Debian gateway-chain class, the zero-parameter/auditable form, and the enumerable released-set reading on AND-semantics graphs (a property of the list, not of the score — see Result 3) — not unique discovery of the two headline packages. Against dependent count and against the incumbent's fame-and-activity signals, the headline rows remain invisible without concentration weighting.
- **Library enrichment is by design.** conemass's head is almost entirely libraries and build plumbing. For growth or importance claims that would be a confound; for supply-chain risk it is the point — libraries and build-time code are the attack surface.
- **The incumbent artifact is the v1-era list.** The v2 pipeline integrates deps.dev dependent counts and might narrow some gaps, though dependent counts are still volume signals and unicode-ident's count is six.[1]
- **Mappings are hand-curated.** Package-to-repository mappings for the join are a table in the published script; errors in it are ours and correctable.
- **The metric is gameable by shell dependents.** A package whose dependency cone is near-empty donates a large per-cone credit to everything it depends on (a cone of size one donates a full 1.0), so publishing shell packages that depend on a target inflates the target's mass. Dependent count is gameable by the same move, but the failure mode should be named rather than discovered: the attack has a detectable signature — mass arriving predominantly from tiny cones — and the same profile occurs naturally as wrapper

restatements in the proof-graph application (`examples/prove2me/REPORT.md` in this repository), where it is filtered by cross-checking transitive dependents.
- **The metric requires bounded closures, and this piece did not say so** (added 2026-10-19). Everything above is measured on package graphs, where a dependency closure is tens to a few hundred nodes and the 200-cap binds for a few percent of cones. On a citation graph (`examples/citations/`: 42,796 papers through 2012, scored against citations received 2013–2020) the closure of almost every paper is the whole field, 79% of cones are truncated, and the ranking is decided by the cap: top-40 overlap with citation count falls 13 → 10 → 5 → 3 as the cap goes 50 → 200 → 800 → 3000, and with citation count and publication period held fixed conemass predicts nothing about later citations (within-cell rho −0.03 to +0.07, the same as PageRank and reach). The head-versus-bulk section carries the anatomy numbers. This is the first external ground-truth test the metric clearly lost, and the loss has a structural reason rather than a statistical one: a closure that is everything collapses the 1/|cone| weight the same way a depth-one star does (the GitHub-issues negative in the same repository), from the other side. Raw citation graphs are out of scope; a windowed or typed-edge variant would be a different metric needing its own validation.
- **A related claim made outside this piece was withdrawn** (2026-10-18). The three-project Alpha-Omega retrodiction in the repository stated that SAST bug yield was flat in conemass rank. At population scale (9,851 audit rows, 4,233-package PyPI graph) it is not: 0 of 20 in the top 1% of the ranking, 16.7% in the bottom half ($p = 0.004$). Load-bearing packages have fewer findings per scan. The piece above makes no yield claim, but a reader who met the metric through that report should know the correction is dated in `examples/alpha-omega/REPORT.md`. The result that held at both scales is that SAST severity is unrelated to reach ($\rho = -0.03$; 11 of 12 High findings in packages at most four others depend on), which is the premise of the CLI's triage mode.

Recommendation, ending on the artifact

Concentration of reach should be a column in criticality dashboards, next to — not instead of — activity-based scores. The two metrics disagree on a specific, enumerable set of packages: quiet, finished, deeply embedded libraries and degree-one build-time plumbing. That set is small (the head of the conemass ranking), cheap to compute for any registry with a dependency graph, and contains the known catastrophic case at rank eight of sixty-three thousand, twenty-one months before anyone knew to look.

Rather than end on the ask, we end on the artifact. github.com/thefalsework/conemass contains the conemass top-1000 for Debian trixie (2025) and crates.io, computed 2026-09-02 and published as-is. The files are dated; every future incident either involves a package in them or it does not, and either outcome scores the metric in public. The CLI beside them runs on any dependency graph in seconds — `node conemass.mjs Cargo.lock --top 50` works directly on a Rust project's lockfile — with no dependencies of its own, under Apache-2.0. The rows to read are the ones where `conemass_rank` is far ahead of `dependents_rank`.

Artifacts and reproducibility

Two artifacts accompany this piece so that its claims can be used, not just checked:

- **Published rankings** (thefalsework/conemass, `rankings/`): dated top-1000 conemass rankings for Debian trixie (2025) and crates.io (2022), with the dependent-count comparison columns inline.
- **A standalone CLI** (same repo, Apache-2.0): a single zero-dependency Node script that takes any dependency graph — a `Cargo.lock` directly, an edge-list CSV of `dependent,dependency` pairs, or a JSON graph — handles cycles, and emits a deterministic ranking. One methods detail disclosed here because it departs from the formula as printed: cycles are condensed before cones are computed, so a dependency cycle hands out **one** unit of credit total (not one per member), counts as one element inside other cones, and its members share a score. A 100,000-node registry takes seconds. Run it on your own graph and inspect the rows where `conemass_rank` is far ahead of `dependents_rank`. The repo's eight-package `test-cargo.lock` reproduces the metric's whole argument in one command: unicode-ident ranks first with two direct dependents, above proc-macro2 with three. Since October 2026 the same file also reads CycloneDX

and SPDX SBOMs and `nix-store -q --graph` output, reports the fraction of cones truncated at the cap (the precondition above), and has a triage mode (`--findings`) that joins a Grype, Trivy or OSV-Scanner report to the graph and prints each affected package with its reach and the number of dependents its fix would clean (`examples/airflow-image/`).
- **Three further external tests**, added after the studies above and recorded in the same repository with their data: the Alpha-Omega audit join at three-project and population scale (`examples/alpha-omega/`), and the citation-graph negative (`examples/citations/`). All three are described in the limitations.

`oracle-scanner/` at github.com/thefalsework/papers also contains the eleven studies behind this piece (retrodiction, cap sweep, crates replication, incumbent join, RustSec retrodiction, head overlap, vintage trajectories, arrival filter, crates arrival, graph anatomy, falsework tightness), their raw outputs, the incumbent CSV as retrieved, and the hand-curated mapping table. Dependency snapshots and their extraction scripts are in `debian-study/` and `software-study/`. Every script's expectations were written in its header before first execution; results are in dated postscripts.

---

[1] The v2 pipeline's published dataset (an `all.csv` on Google Cloud Storage) was unavailable at retrieval time: the bucket returns "the billing account for the owning project is disabled." We used the most recent obtainable artifact of the score as consumed.

[2] R.-H. Pfeiffer, "Identifying Critical Projects via PageRank and Truck Factor," Proceedings of the 18th International Conference on Mining Software Repositories (MSR 2021). https://www.itu.dk/~ropf/blog/assets/msr2021_pfeiffer.pdf

Disclosure. Drafting was AI-assisted under direction.